计算机网络

OSI 七层网络模型：





1. 实现两台机器互联，机器和机器之间怎么通信？--物理层
2. 01010比特流其实是没什么意思的数据，变成有意义的数据呢？，确认这个数据是发给我的呢？---数据链路层
3. 主机A- B，C,D,E- 主机F？？ 路径很多条，怎么选择最优路？ 要怎么知道对方的MAC地址？ --网络层（三层）
4. 发送数据特多，数据包很大，需要很长时间？？中间网络中断，重传？？ 数据包是完整的正确的？---传输层
5. 断点续传功能 --- 会话层
6. 操作系统，windows10，MACOS ,Linux，语法是不一样，不同系统之间进行通信--- 表示层
7. 字节流的格式，不好识别，不好操作。--应用层

ARP、IP、ICMP 的核心关系：

① 如果把网络通信比作一次**物流快递**：

- **IP（网际协议，Internet Protocol）**：相当于**快递单上的收发件人地址（IP地址）**。它负责规划整体的传输路径（路由），把数据包从源主机送到目的主机。
- **ARP（地址解析协议，Address Resolution Protocol）**：相当于**地址翻译官**。IP 寻址使用的是逻辑地址（IP），但在局域网的物理硬件传输中，网卡只认物理地址（MAC 地址）。ARP 的工作就是“通过 IP 地址查对应的 MAC 地址”。
- **ICMP（网际控制报文协议，Internet Control Message Protocol）**：相当于**物流客服与质检员**。它负责在网络出现差错（如目标不可达、路径超时）时发送通知，或者用于网络诊断（如著名的 ping 命令）。

② 工作流程：

1. IP 的工作过程

- 当应用程序发送数据时，传输层（如 TCP/UDP）的数据会交由网络层的 **IP 协议**处理。
- IP 协议会给数据加上一个 **IP 头部**（包含源 IP 地址和目的 IP 地址），形成 **IP 数据报（Packet）**，然后根据路由表决定是直接发给局域网内的设备，还是交由网关（路由器）转发。

2. ARP 的工作过程（如何获取 MAC 地址）

当 IP 决定把数据发给局域网内的某台主机时，会触发 ARP：

1. **查缓存**：主机先查看自己本地的 **ARP 缓存表**，看里面有没有记录“目标 IP 对应的 MAC 地址”。如果有，直接使用；如果没有，发起 ARP 请求。
2. **ARP 请求（广播）**：主机在局域网内大喊一声（发送广播帧，目标 MAC 为 FF:FF:FF:FF:FF:FF）：“谁的 IP 是 192.168.1.100？请把你的 MAC 地址告诉我，我是 192.168.1.1，我的 MAC 是 XX:XX:...”。
3. **ARP 应答（单胞）**：局域网内所有主机都会收到这个广播，只有 192.168.1.100 发现被叫到，它会单独回复（单播）：“我的 IP 是 192.168.1.100，我的 MAC 地址是 YY:YY:...”。
4. **缓存并发送**：源主机收到后，把这个对应关系记入 ARP 缓存表，随后把数据封装成帧发送出去。

3. ICMP 的工作过程（诊断与报错）

- **如何产生**：当 IP 数据包在传输过程中遇到问题（例如：路由器发现 TTL 减到 0、或者找不到通往目标网络的路径、或者目标主机端口关闭），路由器或目标主机就会生成一个 **ICMP 差错报文**。
- **如何传输**：**ICMP 报文并不是独立于 IP 的，它需要被封装在 IP 数据报内部**（在 IP 协议头部中，协议字段标识为 ICMP），像普通数据一样在网络中传输。
- **常见应用（Ping）**：当你执行 ping 命令时，系统会向目标发送一个 ICMP Echo Request（回显请求）报文，目标收到后回复一个 ICMP Echo Reply（回显应答）报文，以此来测试网络连通性和延迟。

③ 与下层关系

在 OSI 模型中，IP 和 ICMP 位于**网络层（第3层）**，而 ARP 则处于网络层与数据链路层的交界处（通常被归类在网络层，但紧密服务于数据链路层）。它们与下层（**数据链路层**和**物理层**）的关系非常紧密：

1. 对数据链路层（第2层）的依赖与封装

- **帧的封装**：网络层的 IP 数据报（或 ICMP 报文）完成后，必须交给数据链路层，加上帧头（源 MAC 和目的 MAC）和帧尾，封装成**数据帧（Frame）**。如果没有下层提供的 MAC 地址（ARP 解决的问题），数据帧就无法在局域网中寻址发送。
- **ARP 的特殊性**：ARP 协议虽然属于辅助网络层的协议，但它的报文**不经过 IP 封装**，而是直接交由数据链路层打包成以太网帧发送（其以太网类型字段为 0x0806）。可以说，ARP 是直接架在数据链路层之上的。

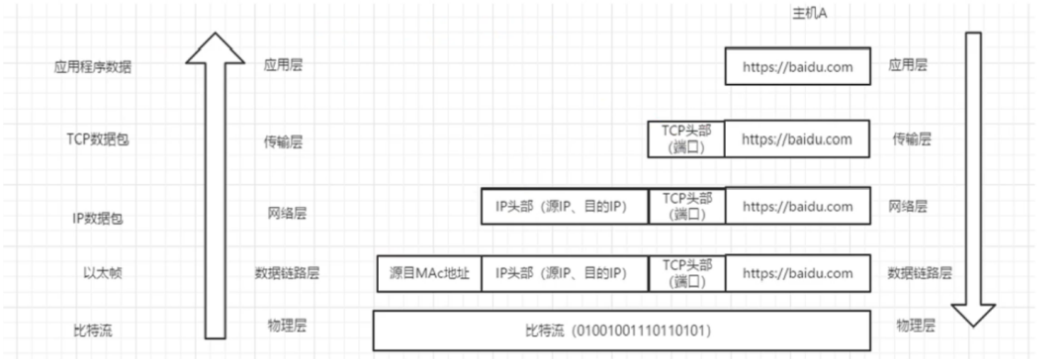
2. 对物理层（第1层）的最终承载

- 无论是 IP 转发的数据包、ARP 的广播请求，还是 ICMP 的控制报文，当它们在数据链路层被组装成完整的“帧”之后，最终都要流向**物理层**。
- 物理层负责将这些高层的二进制比特流转换成**物理信号**（如网线中的电信号、光纤中的光信号、无线网卡发射的电磁波），通过真实的物理介质传递到对端设备。

协议	核心作用	OSI 层级	与下层的关系
IP	逻辑寻址、路由转发	网络层 (第3层)	将数据包下交给数据链路层封装为帧，依赖物理层传输
ARP	IP地址 -> MAC地址转换	网络层 / 链路层接口	直接使用数据链路层的广播机制发送请求
ICMP	差错报告、网络诊断 (Ping)	网络层 (第3层)	封装在 IP 数据报内，最终同样依靠下层完成实际传输

数据传输过程：

1. 封装
2. 解封装



ARP 协议(IP 解析 MAC地址)：在同一个局域网下，使用MAC地址通信，通过ARP(通过ARP表查询)获取主机MAC地址，建立两机通信

IP 协议(跨网段通信)：在不同局域网下，两台主机不能直接通信，使用IP地址通信，通过IP(通过路由表查询，每个IP出局域网都要通过一个网关，根据网关的路由表查询IP连接路径)查询连接路径并获取其目标IP地址，达到另一个局域网，通过ARP协议解析目标IP地址，从而得到另一个局域网的MAC地址，再返回给源地址，从而实现不同局域网通信

网络层协议：

ARP协议：

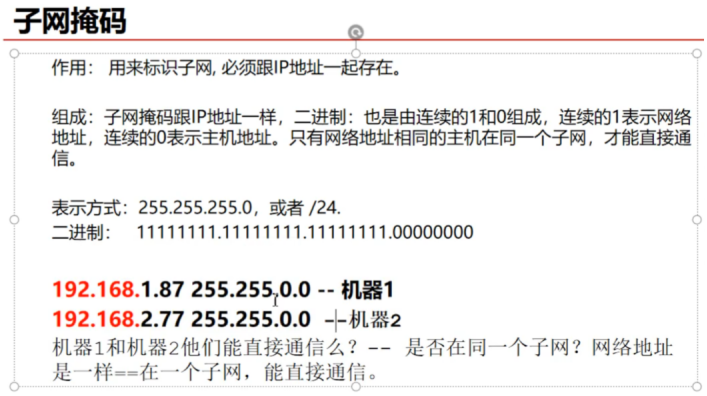
IP地址 -> MAC地址转换(直接使用数据链路层的广播机制发送请求)

IP 协议：

IP 地址 = 网络号 + 主机号



子网掩码：



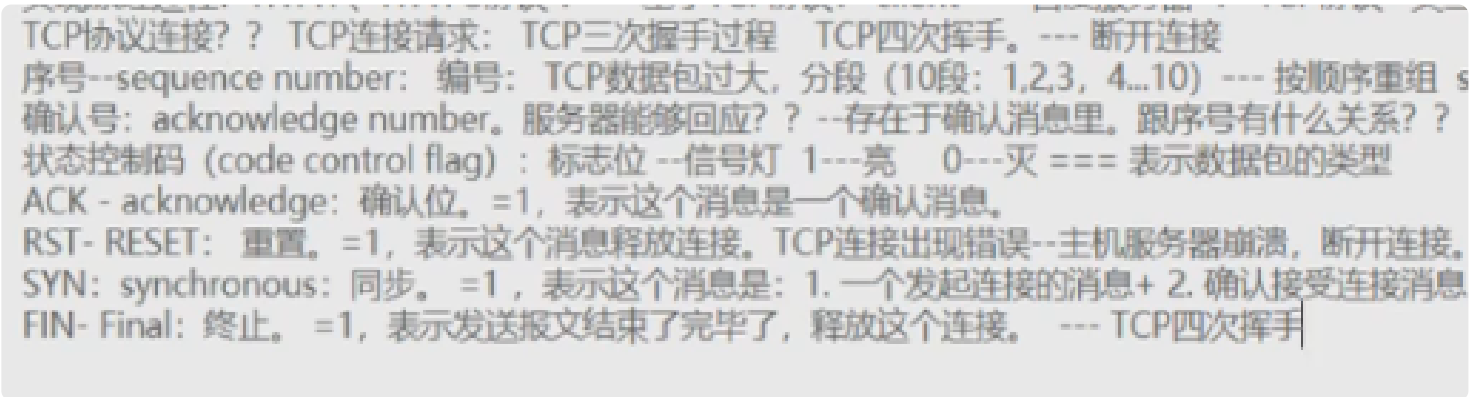
路由协议(一组路由协议 查找路由表)：

静态路由：目的地址 --- 指定下一跳(即 下一个网关是谁) --- 默认路由(一个特殊的静态路由)

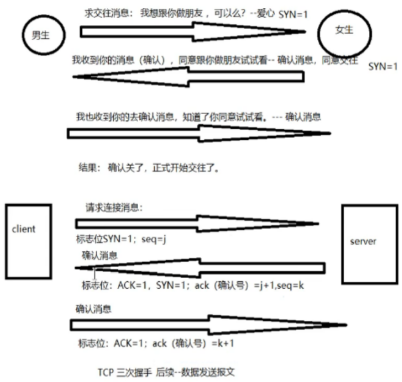
动态路由：配置之后，动态学习路由条目 --- 路由表 ---> RIP OSPF BGP

传输层协议：

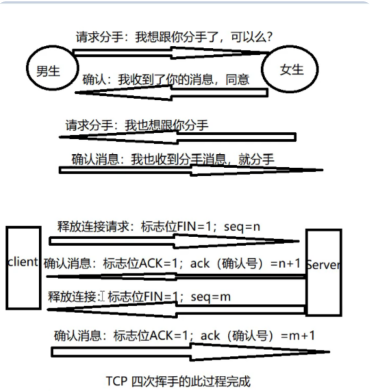
TCP协议(传输控制协议　有连接)：



TCP三次握手(“一次请求，两次确认”)：



TCP四次挥手（“两次请求，两次确认”）：



协议：HTTP、HTTPS、FTP

UDP协议(用户数据报协议 无连接 不可靠)：

使用场景：

1. 不需要可靠机制，只要求速度快
2. 流媒体、多媒体游戏、IP电话
3. 资源消耗特别小

协议：DNS、SNMP、DHCP、RIP

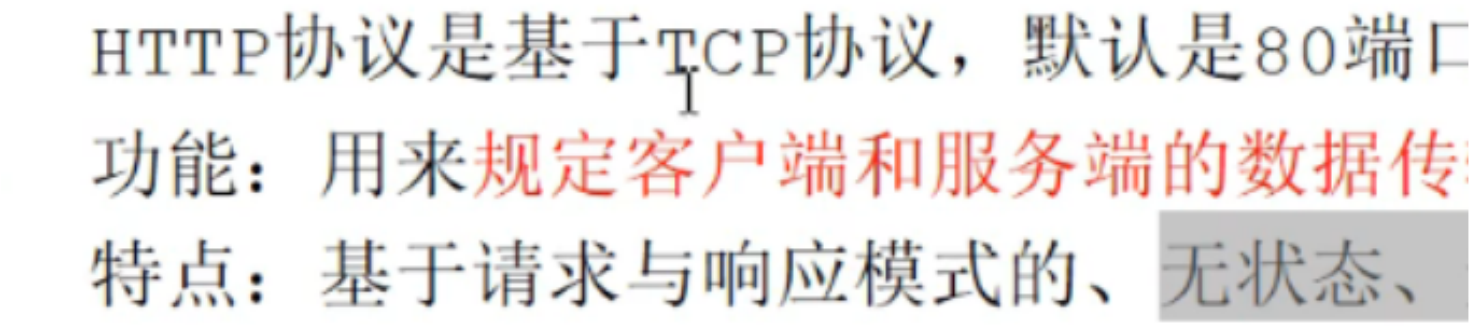
TCP与UDP的对比：

特征点	TCP	UDP
是否连接	面向连接	面向非连接
传输可靠性	可靠	会丢包，不可靠
应用场景	传输数据量大	传输量小
速度	慢	快

1. TCP是面向连接的，所有TCP三次握手和四次挥手的过程；UDP无连接协议；
2. TCP因为有可靠的连接机制== 可靠协议； UDP没有连接和确认机制，==会丢包，会出错，不可靠的协议；
3. TCP协议：数据量很大，防止丢包，正确重传；对数据正确性很关心； UDP：量小
4. 速度：TCP协议速度很慢，传输效率低； UDP：速度很快，传输效率很高。

应用层协议：

HTTP协议(超文本传输协议):



请求/响应行：请求报文是“请求行”（方法、URL、HTTP版本）；响应报文是“状态行”（HTTP版本、状态码、原因短语）。

请求/响应头：若干头部字段，如 Host、Content-Type、Content-Length 等。

空行：首部与主体之间的空行，用于分隔

请求/响应体：实际传输的数据，如请求体或响应体，可为空。

状态码：

```
HTTP/1.1 200 OK  -- 响应行：HTTP版本 + HTTP状态码
状态码：1xx - 接收的请求正在被处理
        2xx - 请求正常处理完毕 200
        3xx - 重定向：302, 304缓存 --没有改动，不处理
        4xx - 客户端的问题导致的错误：网址错误- 404
        5xx - 服务器问题：503
```

HTTP 无连接 特点：

限制每次连接只处理一个请求

服务器处理完客户请求，并收到客户应答后，即断开连接

Keep-Alive设置：

1.开启：Connection: keep-alive ，会发起Keep-Alive的连接请求--长连接。HTTP1.1-默认打开的。--不需要重新建立连接。

2. 关闭：在http头中设置Connection: close，即可关闭。

3.设置连接时间：

在http header中设置Keep-Alive: timeout=5, max=1000

- timeout是超时时间，单位秒，超过这个时间后就断开连接 --释放
- max是最多的连接次数，若超过这个次数就强制断开连接

HTTP 无状态 特点：

对事物处理没有记忆能力，服务器不知道客户端是什么状态；给服务器发送HTTP请求之后，服务器回应之后，不会有任何记录

每个请求都是独立的

服务器处理后续请求-前面请求的信息 -- 重传。

优点：释放了服务器压力

缺点：重传--重复内容，增大数据量--- 资源浪费

动态交互：淘宝：购物车 --- 商品信息； login--后续

cookie和session：

- Cookies：将前面请求信息保存成一个临时文件 --- cookies值。---存放在浏览器里

淘宝购物车：不同商品信息写进入cookies；

登录：充值：login--》充值。可以把登录的账号信息--- cookies值。

- Session：永久的cookies值。session ID（会话编号）---保存服务器上，--传递给客户端，保存在内存里。

问题：资源占有。session超时时间。清除session ID。

HTTPS协议(超文本传输安全协议)：

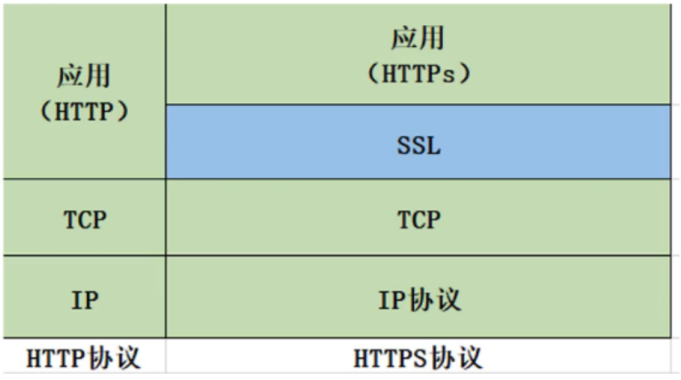
概念：HTTPS（Hypertext Transfer Protocol Secure),超文本

HTTP通道，简单讲是HTTP的安全版。它加密数据并确保其机密性

网站交互时免于窃取个人信息和计费数据---身份认证；防止信息被

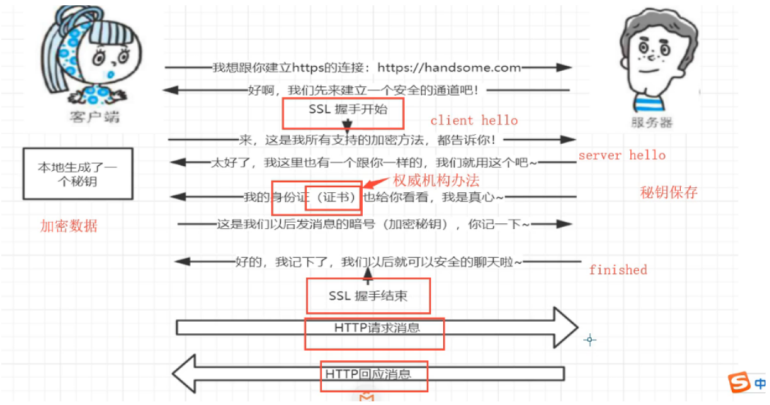
披着SSL外壳的HTTP协议！ ---SSL协议来实现。

HTTP + 加密 +身份认证+完整性保护 = HTTPS---过程和原理？？



加密--- 秘钥； 身份认证-
法--计算一个hash值，验证

SSL协议及握手：



1.客户端请求建立SSL连接，并将自己支持的一套加密规则发送给服务器。

2.网站从中选出一组加密算法与HASH算法,并将自己的身份信息以证书的形式发回给浏览器。证书里面包含了网站地址，加密公钥，以及证书的颁发机构等.
一对钥匙：公钥--公布--加密； 私钥--私有，解密的

3.获得网站证书之后浏览器要做以下工作：
Ø 验证证书的合法性-服务器身份，如证书受信任，浏览器会生成一串随机数密码，并用证书中提供的公钥加密。
Ø 把密码发送给网站。
Ø 告诉网站后续报文要加密了。握手结束。

4.网站接收浏览器发来的数据之后要做以下的操作：
Ø 使用自己的私钥将信息解密取出密码 -- 加密了
Ø 告诉客户端后续报文要加密了。握手结束。

HTTP vs HTTPS:

	HTTP	HTTPS
安全性	明文传输，易受攻击；无法确认双方身份，也无法保证数据的完整性-安全性低	使用ssl加密传输协议，信息是密文，可以认证双方的身份，防止信息被截取篡改--安全性高
TCP端口	TCP 80端口	TCP 443端口
灵活度	简单快速，使用很灵活	技术门槛较高，多数个人或私人网站难以支撑
速度	协议简单，HTTP服务器的程序规模小，因而通信速度很快。	加重了服务端的负担，需要更多的资源来支撑，降低了用户的访问速度
经济适用度	没有额外的费用要求	CA机构颁发的证书都是需要年费的，此外对接Https协议也需要额外的技术支持